Physics of Coding Agents: Part 1

Fit, Not Format at the Edit Interface

Langlin Wan[1] Zhixuan Liang[2] Guangyuan Wang[3,4] Shilong Liu[2,5,*]

[1]Carnegie Mellon University [2]Princeton University [3]McGill University
[4]Mila – Quebec AI Institute [5]Columbia University

[*]Corresponding author: Shilong Liu, slongliu86@gmail.com









Code & data https://github.com/WanLanglin/fit-not-format-edit-harness
Harness https://github.com/sai-foundation/s-code

October 2026 · version 1

Abstract

In a coding agent, the edit interface is where model and harness meet: to edit a file, the model must say where a change goes, by pointing at line numbers or by quoting existing code, as search/replace and patch formats require. Models become fluent in both forms from internet text in pretraining and from harness trajectories in post-training, and an interface fits a model when it asks for the forms the model writes reliably.

In a controlled harness where only the edit interface varies, across six models, three benchmarks and more than 4,700 runs, we find that task success hides a poor fit. On large-file refactors, GPT-5.6-luna's line-range edit is accepted on the first attempt in 99% of runs, compared with 58% for search/replace, yet both solve the same share of tasks. Agents retry rejected edits, so the gap shows up as token cost. The interfaces also fail differently. A long quotation costs output and fails visibly, while a line range is cheap but a wrong one silently edits the wrong lines, most often off by a single line. Which form wins depends on the model, and given several formats, models reach for their habitual form rather than the one they write most fluently.

Building on the two forms that natural text teaches every model, we introduce VERIFIED SELECTION, in which the model gives a line range and quotes only its first and last lines, while the harness checks them and absorbs small offsets. On large-file refactors, compared with line ranges, it cuts silently broken first edits by 63% for GPT-5.6-luna and 92% for MiMo-v2.5-Pro; compared with search/replace, it solves 1.43–1.77× as many tasks per output token. On Microsoft's multi-file RefactorBench, it cuts broken first edits from 11 to 2. **The lesson is fit, not format: models should express edit intent in a compact form they produce reliably, while harnesses handle verification and bookkeeping.**

Keywords: coding agents, LLM agents, code editing, edit format, search/replace, diff, line numbers, tool calling, agent harness, agent-computer interface, token efficiency, refactoring.

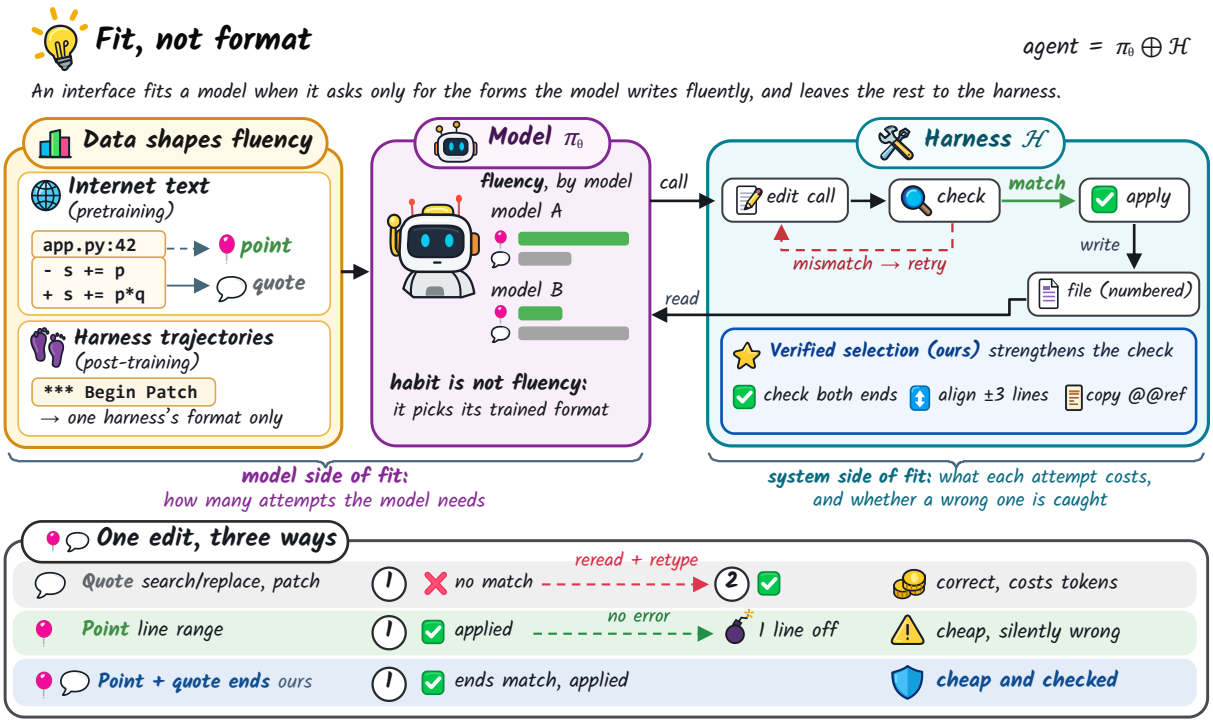


Figure 1: Fit, not format. In the top row, natural text and harness trajectories give a model fluency in pointing at code and in quoting it, to degrees that differ between models (the bars are schematic), and a habit of reaching for its training format. The model sends an edit call, and the harness checks it before writing the file; a mismatch is returned and the model retries. VERIFIED SELECTION strengthens the check by comparing both boundary lines with the file, aligning small offsets and copying lines referenced with `@@ref`. Fit has a model side, how many attempts the model needs, and a system side, what each attempt costs and whether a wrong edit is caught. The bottom row states one edit three ways. A quotation that does not match is rejected and retyped, which costs output; a line range is cheap, but a range that is off by one line is applied without any error; VERIFIED SELECTION points and quotes only the two boundary lines.

1 Introduction

On large-file refactors, GPT-5.6-luna's first edit is accepted in 99% of runs when it points at the lines to change and in 58% when it quotes them, yet both ways solve 94.4% of the tasks.[1] This paper explains why such a gap stays hidden, and what it costs. A coding agent changes code by sending edit calls to its harness, which checks each call and applies it to the file. Suppose the agent must change lines 119–125 of a long file. It can quote those seven lines back to its harness, which replaces the text it finds, or it can point at them by number, and the harness replaces those lines without looking at them. The quotation costs output and is rejected if it does not match the file. The pointer is cheap, but a range that is off by one line changes the wrong code without any error. Existing harnesses take different sides. Claude Code's edit tool and Anthropic's text-editor tool take an exact quotation of the old text [3, 5]. OpenAI's `apply_patch` locates changes by context lines and explicitly avoids line numbers [27, 28], while SWE-agent's original interface replaced a

[1] Numbers in the introduction come from Table 1 (all tasks, first run). Tables 2 and 3 use only the tasks valid in every method arm, so their values differ slightly (for example, .60 instead of .58).

numbered line range [35]. The choice fixes how much of the target the model must restate and what the harness can check before it writes (Figure 1). It is part of the agent's execution mechanism, not a serialization detail.

The choice is not neutral for the model either. People refer to code both ways (a reviewer quotes the offending line or cites it by number), and so does the internet text that models are pretrained on (Section 2.1). Post-training adds a second source, since developers train models on trajectories generated in their own harnesses. OpenAI publishes a diff format "on which the model has been extensively trained" [28], Anthropic "publishes the schema and trains Claude on it" [4], and a harness vendor reports that an unfamiliar format "costs extra reasoning tokens and produces more mistakes" [19]. According to a recent survey, an agent is a policy coupled at run time to a harness, Agent $= \pi_\theta \oplus \mathcal{H}$ [12]. Developers' own reports indicate that the formats in the harnesses and text a model is trained on can become part of how the model acts [8, 32]. From both sources a model becomes *fluent* in two ways of addressing code, pointing at it and quoting it, and forms a *habit*, the form it reaches for when it may choose. A model may be more practiced in one of these ways of referring than in the other. Asked to refer in a way it has rarely practiced, it pays as a person would, by restating, retrying and correcting itself, and the harness bills every attempt. Natural text shapes fluency that every model shares, whereas post-training on one harness adds fluency in that harness's format alone. The edit interface is where fluency and habit meet the harness, and we ask one question:

When a model edits code through a harness, do the cost of an edit and whether a wrong one is caught depend on the model, the interface, or the pair?

Our answer is *fit, not format*: no edit format is best on its own. The format sets the price of each attempt and whether a wrong one is caught; how fluently the model writes it sets how many attempts it needs. Fit covers both, and the cost of a misfit hides in the agent loop.

Task success, the usual yardstick, misses most of this. An agent keeps editing until the model judges the task done, so a rejected edit usually becomes a retry rather than a failure, and a silently wrong one may never be noticed. Instead, the differences between interfaces show up in the first action, in the cost of retries, and in which errors are seen at all.

The evidence comes from a fixed harness in which only the edit interface varies, across six models, three code-editing benchmarks and over 4,700 runs, with action-level replay of first edits. We find:

1. **Success hides misfit; cost reveals it.** The gap that opens this paper is typical. The loop retries rejected edits, so the difference goes into cost instead. A run whose first edit is rejected costs up to 1.8 times as much, and on the refactors GPT-5.6-luna solves 1.64 times as many tasks per output token with line ranges as with search/replace (Section 4). Interfaces should be compared by first-action acceptance and tasks solved per output token, not by success alone.

2. **Evidence makes errors visible: quoting is checkable, pointing is cheap but silent.** The longer a quotation, the more often it fails to match, but the restatement lets the harness check what the model asks it to change. Line ranges save that output, and save more the longer the span, but a wrong range is applied without any error. About one in five large first line-range edits is accepted and leaves code that no longer parses, and most of these would parse if one boundary moved by a line. Edits that land in the wrong place but still parse are not even included in this count (Sections 5 and 8).

3. **Fit, not format, and habit is not fluency.** Whether first edits are accepted depends on the pairing of model and interface (a significant model × interface interaction), even between search/replace and line ranges, which differ only in whether they point or quote. Each of the three interfaces is the best-accepted one for at least one model. Given a choice, DeepSeek-V4-Flash reaches most often for the form it writes worst, GPT-5.6-luna picks the patch format its developer trains on more than twice as often as line ranges, which it writes best, and some favor the tool listed first (Section 6).

Read together, the results describe a division of labor that follows from fit. The model states its intent in the forms it writes fluently (Result 3). The harness does the bookkeeping of checking the target, reconciling small offsets and copying code, which is exact and nearly free for a harness but costly and error-prone in the model's output (Results 1 and 2).

We then build VERIFIED SELECTION from pointing and quoting together (Figure 1; call syntax in Figure 2d,e). The model points with line numbers and quotes only the first and last line of the span. The harness checks those boundaries, where silent errors concentrate, absorbs consistent small offsets, and copies already-read lines that the model references with a `@@ref` marker. On the refactors, over two runs and compared with bare line ranges, it cuts broken first edits by 63% for GPT-5.6-luna and 92% for MiMo-v2.5-Pro; compared with search/replace, it solves 1.43–1.77 times as many tasks per output token. On Microsoft's RefactorBench, where one edit call often changes several files, it cuts broken first edits from 11 to 2. Its quoted boundaries also expose what a bare pointer misses, since 16% of first edits give line numbers that contradict their own quotes and a third of these would have parsed as bare ranges, invisible to any parse check. For Claude Sonnet 5, whose developer documents training it on string replacement, VERIFIED SELECTION is accepted at least as often as that trained format, although no model was trained on VERIFIED SELECTION (Section 8).

2 Fluency and Fit

2.1 Fluent in two ways of addressing

We call an edit *position-addressed* when its target is named by coordinates in a file the model has read (a line range), and *content-addressed* when the target is named by reproducing its text (an exact quotation, or patch context that must match). For brevity, we say that a position-addressed edit *points* and a content-addressed edit *quotes*. The distinction concerns the edit call, not how the model found the code. In both cases the model has usually read the file first.

Both forms are common in the internet text that models are pretrained on. Position references pervade developer artifacts. Compiler diagnostics conventionally begin with `sourcefile:lineno:` [13], Python tracebacks report the line of each frame [30], and code hosts link to single lines and line ranges [17]. Content references are the substance of version-control history. Commit diffs identify changes by context lines and are used at scale to train code models [26]. Text also shows why pointing is cheap. Once something is in view, writers refer to it ("the function above", "lines 120–150") instead of restating it. Pointing draws on that fluency and costs few tokens, while quoting draws on the fluency in diffs and makes an edit checkable.

Post-training adds a second source of data, the trajectories that a model generates while using a harness, which carry that harness's edit format. The model developers whose documentation we could verify train on their own edit formats. OpenAI models are trained on a patch format that quotes context lines [28], and Anthropic models on exact string replacement [3, 4]. Open-weight developers train in specific scaffolds too (the code-agent RL scaffold of MiMo-V2-Flash

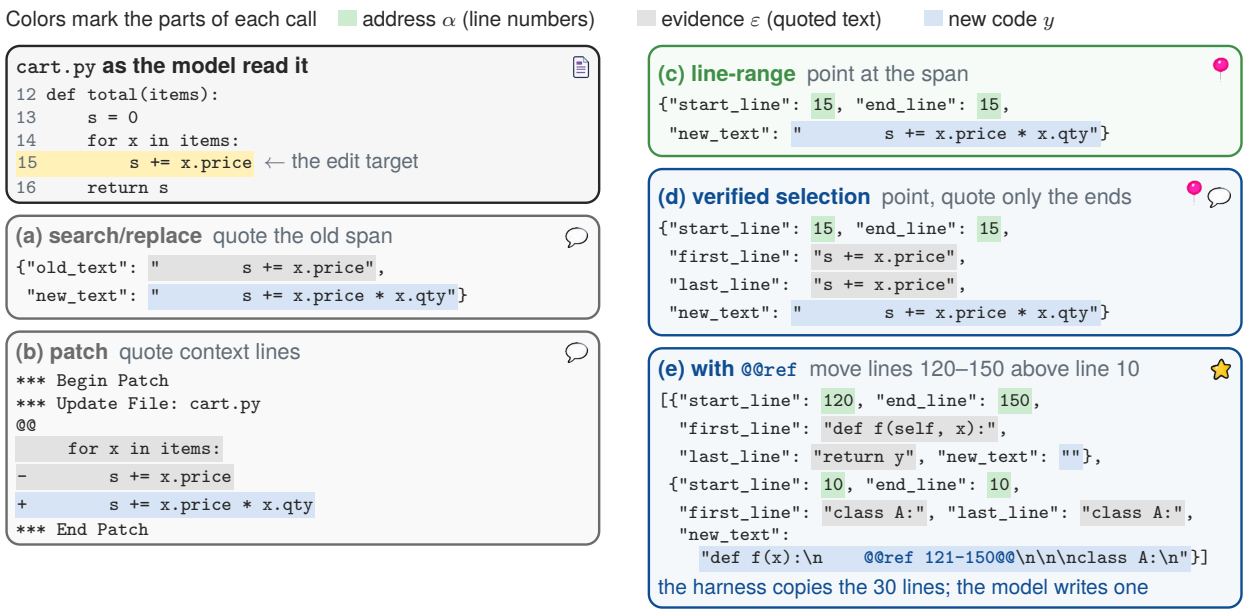


Figure 2: One edit in four interfaces. Line 15 of `cart.py`, `s += x.price`, becomes `s += x.price * x.qty`. Colors mark the parts of each call, its address (green), the evidence of the target it quotes (gray) and the new code (blue). Quoting interfaces (a, b) restate the target and let the harness check it, while the pointing interface (c) restates nothing, so the harness cannot check it. VERIFIED SELECTION (d, e) points with line numbers, quotes only the two boundary lines, and can reference already-read lines with `@@ref` instead of retyping them.

exposes a `str_replace` tool [34]), and models trained on one scaffold's trajectories transfer poorly to others [8, 32]. The format itself is learned. Supervised fine-tuning raises a base model's share of correctly formatted search/replace edits from 12% to 96% [33], and harnesses that serve many models configure the edit format per model [16, 19]. We call what a model has learned about each form of address its priors, in the Bayesian sense, and keep two of their effects apart. A model's *habit* is the form it reaches for when it may choose, and its *fluency* is how reliably it writes each form. Both are behavioral and measured at the interface. The two sources differ in reach. Natural text gives every model some fluency in both forms, while post-training on a harness adds fluency in that harness's format, which models trained elsewhere lack. Because developers' harnesses and data mixes differ, this account predicts that an interface's fit should depend on the model.

2.2 Edit calls and the harness check

An edit call is a triple

$$a = (\alpha,\ \varepsilon,\ y), \tag{1}$$

an *address* α, target *evidence* ε, and replacement text y. Before writing, the harness checks the call against the current file, and what it can check depends on the interface. In SEARCH/REPLACE, ε is the old text and α is derived from it (the unique match). In PATCH, ε is the context and α is again derived. In LINE-RANGE, α is a line range and ε is empty, so the check can verify only the file revision and the bounds of the range. A rejection is feedback the model can act on. An accepted edit whose span is not the intended one changes the file without any feedback, so the error stays hidden until something else reveals it.

A harness can tell that an address misses its intended target only if it has evidence of that target, and in a single edit call that evidence is ε. Omitting it from the call removes its cost, not the need for it.

We call an edit *accepted* when the check passes, and *silently wrong* when it is accepted but changes the wrong code, which the check cannot see. An interface fits a model when it asks only for forms the model writes fluently and leaves the rest to the harness. If models and interfaces combined additively, an interface would help or hurt every model alike. A model $\times$ interface interaction in acceptance is therefore the signature of fit as a property of the pair, which Result 3 tests.

2.3 What misfit costs in the loop

The output an edit call requires grows with the text it restates, which is the new code plus the evidence. Moving a block shows how far apart the interfaces can be. SEARCH/REPLACE writes it twice (quoted to delete it and again to insert it), LINE-RANGE writes it once, and an interface that can reference already-read lines writes only a one-line marker.

If the agent retries until an edit is accepted, and each attempt costs its output plus one pass over the current context, the expected cost of an accepted edit is the cost of one attempt divided by the acceptance rate. How many accepted edits are wrong depends on what the check can see. The price of an attempt and what the check can see are the system side of fit; the acceptance rate is its model side. The results follow this split. The loop turns a low acceptance rate into cost (Result 1), the evidence an interface asks for trades output against silent errors (Result 2), how often each form goes through depends on the model (Result 3), and one design can address all three at once (Result 4).

Evidence as a checksum. Target evidence works like a checksum on the address. A line range carries none. A full quotation repeats the whole target, the most expensive form of redundancy. Quoting only the two boundary lines places a small checksum where pointing errors occur, at the edges of the range (Section 5). A revision hash plays the same role for a stale view of the file.

Reading is cheap, but writing and re-reading add up. Transformer inference processes the input in a parallel, compute-bound prefill and generates output one token at a time in a sequential, memory-bandwidth-bound decode [40]. An agent's context is append-only, so serving systems reuse the key–value (KV) cache of the shared prefix across calls [39], and providers bill cached input for current models at a tenth of the uncached price or less [2, 29]. Under perfect prefix caching, a run therefore pays full price only for what is newly read and what is written (Equation (2), Appendix A). Per token, writing is the expensive part. Per step, the re-read prefix can dominate. In Claude Code and Codex sessions, a typical model call re-reads about a hundred thousand cached tokens to write about two hundred, and cached reads dominate total API cost [41]. Every retry pays one such re-read.

3 Setup

Harness. All runs use S-Code (`https://github.com/sai-foundation/s-code`), an open-source terminal coding-agent harness. The agent has three read-only tools (`list_files`, `search_text`, and `read_file`, which returns numbered lines and a file revision) and one edit tool, so that every correction it makes passes through the edit interface under study. Every write must carry the

current revision. A run ends when the model stops or reaches fixed limits on model calls, tool calls and tokens (Appendix A). Only the edit interface varies within a comparison. The system prompt, read tools, limits, and tasks are fixed, and prompt and schema hashes are logged for each run.

Interfaces. SEARCH/REPLACE takes `old_text` and `new_text`, and the old text must occur exactly once. PATCH takes a patch in the style of Codex's `apply_patch` [27], with add and update operations and context that matches exactly once. LINE-RANGE takes `start_line`, `end_line` and `new_text`, with numbers referring to the file as last read. Every arm also permits a whole-file write, which is recorded but not counted as a targeted edit. SEARCH/REPLACE and LINE-RANGE share the same call structure and instructions and differ only in the fields that address the target, so comparing them isolates pointing against quoting. For Result 3, menu conditions offer all three forms, either as argument shapes of one tool or as three named tools, with and without advice on when to use each.

Models and benchmarks. The main study uses GPT-5.6-luna, GLM-5.3, DeepSeek-V4-Flash, MiMo-v2.5-Pro and Qwen3.8-Flash, each through one pinned provider with its documented sampling settings (Table 5). CanItEdit [9] gives a file, an instruction and hidden tests. Its files are short and most edits replace a few lines. We use the 82 tasks that run without third-party libraries. The Aider refactoring benchmark [1] (*the refactors*, for short) has 89 refactoring tasks in long real-world files, scored on the syntax tree. All five models run the three interfaces on CanItEdit, which measures fit. On the refactors, where costs and silent errors grow with edit size, two models with opposite fits run every interface and every ablation, and SEARCH/REPLACE, LINE-RANGE and VERIFIED SELECTION a second time. On CanItEdit, line ranges are accepted most often for GPT-5.6-luna and search/replace for MiMo-v2.5-Pro. Runs that fail because of the provider or the harness rather than the model are excluded (Appendix A), and every paired comparison uses the tasks with a valid run in every arm. Claude Sonnet 5 joined later on CanItEdit with SEARCH/REPLACE and LINE-RANGE only (Appendix A). The method evaluation (Result 4) adds VERIFIED SELECTION and its ablations for GPT-5.6-luna and MiMo-v2.5-Pro on both benchmarks, and VERIFIED SELECTION for Claude Sonnet 5 on CanItEdit. To test VERIFIED SELECTION on edits that span more files and make more complex changes, we add RefactorBench [14], Microsoft's benchmark of multi-file refactorings in nine open-source Python repositories. We run all four arms with GPT-5.6-luna on its 84 tasks that need no file deletion (Appendix E).

Outcomes. We measure outcomes at three levels, the acceptance of the first targeted edit (the *first edit*, for short), recovery through later edits or whole-file writes, and final task success. To measure visibility we distinguish rejected calls from accepted calls after which the file no longer parses. High acceptance is good only when the accepted edits are right, so we read acceptance together with silent error. An interface that accepts every edit but applies some to the wrong lines is not a better fit. All three benchmarks are in Python, and we parse with Python's own parser. We call an accepted edit after which the file no longer parses *broken*. Breakage is the part of silent error that leaves a syntactic trace. It bounds silent error from below, since an edit that changes the wrong lines but still parses is not counted. The VERIFIED SELECTION arms, whose calls quote the lines they point at, expose such edits when numbers and quotes disagree (Section 8). For every first targeted call in the main study whose address can be interpreted, we replay the action on the original file, measure the replaced region in token-equivalents (characters divided by four), and parse the result (Appendix A). Efficiency is tasks solved per output token (reasoning included) and per dollar, always within a model. Because a provider's cache hit rate varies with the date of a run,

Table 1: Main comparison, in the order SEARCH/REPLACE / PATCH / LINE-RANGE. p: Cochran's Q on task success ([§]exact McNemar for Claude Sonnet 5, which has no PATCH arm). Output is thousands of tokens per task (reasoning included), comparable only within a row. First edit accepted counts runs that make a targeted edit.

Benchmark	Model	Tasks	Task success	p	First edit accepted	Output / task
CanItEdit	GPT-5.6-luna	82	.756 / .780 / .805	.34	.93 / .23 / .99	1.1 / 1.5 / 1.3
	GLM-5.3	82	.817 / .793 / .768	.30	.97 / .89 / .96	11.2 / 11.9 / 11.8
	DeepSeek-V4-Flash	82	.732 / .683 / .659	.21	.84 / .25 / .90	4.5 / 5.2 / 4.4
	MiMo-v2.5-Pro	82	.780 / .817 / .829	.31	.93 / .16 / .87	1.6 / 2.1 / 1.6
	Qwen3.8-Flash	79	.823 / .861 / .848	.42	.66 / .86 / .45	7.4 / 8.4 / 8.0
	Claude Sonnet 5	80	.787 / – / .825	.38[§]	.92 / – / 1.00	2.3 / – / 2.2
Refactor	GPT-5.6-luna	89	.944 / .944 / .944	1.00	.58 / .33 / .99	4.3 / 4.5 / 2.5
	MiMo-v2.5-Pro	88	.932 / .841 / .841	.093	.76 / .19 / .80	4.3 / 8.7 / 3.6

we report dollars both as billed and under perfect prefix caching (Equation (2)). Output-token efficiency is our primary cost measure because it does not depend on caching.

Statistics. We report Wilson 95% intervals for proportions, Cochran's Q across three paired arms, and exact McNemar tests between two. Efficiency ratios use paired-task bootstrap intervals, and an asterisk marks intervals that exclude 1×. Edit-level regressions resample whole tasks.

4 Result 1: Success Hides Misfit, Cost Reveals It

Where did the difference in the opening example go? Because the model, not the harness, decides when an edit task is done, a poor fit need not change the outcome. It changes what the loop spends to reach it, since every rejected attempt is paid for and then repeated.

Table 1 and Figure 3 separate the first action from the final outcome. First-edit acceptance varies widely across interfaces, while task success on CanItEdit differs by 4 to 7 percentage points within a model, never significantly. GPT-5.6-luna's first patch is accepted in only 23% of CanItEdit runs, yet with patches it solves 78% of the tasks.

The agent loop explains the gap. A rejected edit returns an error the model can act on. The model reads the error, re-reads the file if needed, and tries again or falls back to a whole-file write. Recovery thus keeps task success close across interfaces, but not cost. On matched tasks, runs whose first targeted edit is rejected cost 1.2–1.8 times as much as runs whose first targeted edit is accepted (Figure 4c).

The large-file refactors give the clearest case. Moving from CanItEdit's short files to the refactors' long ones, acceptance of GPT-5.6-luna's first SEARCH/REPLACE edit falls from .93 to .58 while that of its first LINE-RANGE edit stays at .99 (Figure 3a). All three interfaces solve the same share of tasks for GPT-5.6-luna, but its first line-range call is about half as long as its first SEARCH/REPLACE call. With line ranges it solves 1.64 times as many tasks per output token over two runs (Table 3; first run in Figure 4a). Where a model's first calls often fail, retries dominate instead. On the refactors, MiMo-v2.5-Pro writes about twice as much output with patches as with SEARCH/REPLACE and solves fewer tasks, at 0.45 times the efficiency. An interface that writes less is cheaper only if the model can use it.

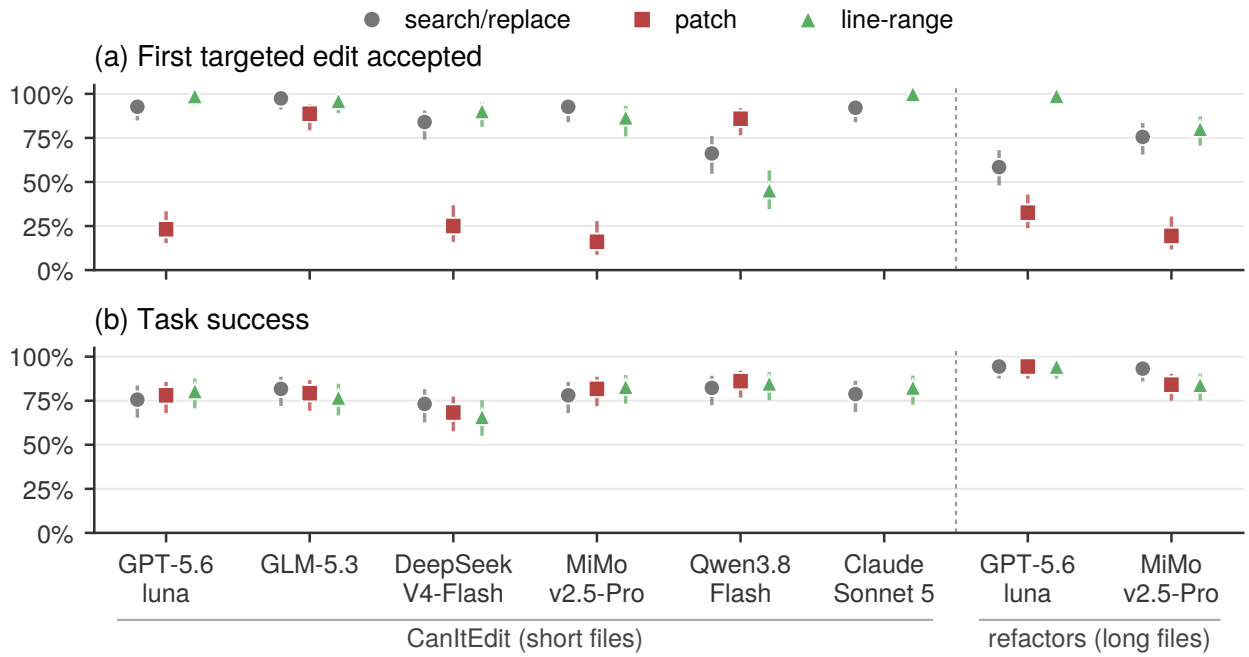


Figure 3: First targeted-edit acceptance (a) and task success (b) for each interface, on CanItEdit (six models) and the large-file refactors (two models). Error bars are Wilson 95% intervals, and interfaces are paired by task within each model. Acceptance spans .16–1.00, while success varies far less.

Where the money goes. Our benchmarks' contexts are short, so under perfect prefix caching output is 33–69% of a run's cost. In long real sessions the balance shifts toward re-reading [41], which makes each avoided retry worth more (Appendix C).

Result 1 (Figures 3 and 4). *Success hides misfit; cost reveals it:* the agent loop turns failed edits into retries, so interfaces differ in what a task costs, not in whether it is solved.

Why does one interface cost more than another to get right? The next section looks at what each form makes the model write, and what that buys.

5 Result 2: Evidence Makes Errors Visible

This section turns to the system side of fit, asking what each form costs to write and whether the harness can catch a wrong one. Quoting puts the evidence for a check in the model's output, while pointing leaves the harness nothing to check.

Quoting costs output and doubles as verification. Across the SEARCH/REPLACE edits of the main study, doubling the length of the quoted old text raises the odds of a no-match rejection by about a quarter, conditional on new-text and file length, whereas doubling the new text does not (Appendix B). Longer quotations do make ambiguous matches rarer (Figure 5). The asymmetry points to an exact-reproduction burden specific to the target, not a general penalty for writing more. The same quotation is what lets the harness refuse a mismatched edit before it changes the file. Restating the target is an explicit verification, paid for in output.

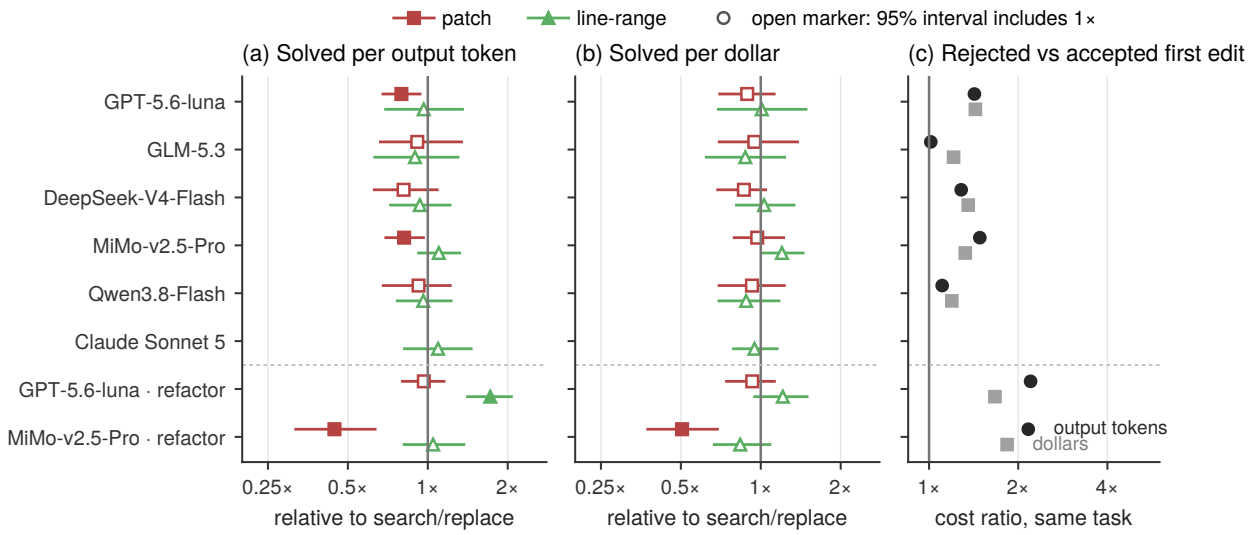


Figure 4: Efficiency of PATCH and LINE-RANGE relative to SEARCH/REPLACE within each model, on paired tasks, as (a) tasks solved per output token and (b) tasks solved per billed dollar (all arms of a model ran in the same batch), with paired bootstrap 95% intervals (open markers mean the interval includes 1×). (c) Output tokens and dollars of runs whose first targeted edit was rejected, relative to runs where it was accepted, on matched tasks.

Pointing is cheap and silent. For edits replacing fewer than 200 token-equivalents, which dominate the sample, about 95% of both LINE-RANGE and SEARCH/REPLACE calls apply and leave a file that parses, so the compact address costs nothing in clean application. For larger regions the failure modes separate (Figure 6). SEARCH/REPLACE rejects about one large edit in eight and silently breaks about one in fifty. LINE-RANGE is rejected only about 1% of the time, but about one large edit in five applies and breaks the file. Most broken first LINE-RANGE edits would parse if one boundary moved by a single line, exactly the kind of error a quoted boundary line detects.

One silent error. In a RefactorBench run, GPT-5.6-luna adds a parameter in four files with one line-range call. In `celery/apps/multi.py` it replaces lines 44–51, but its new text ends one line early and drops the closing parenthesis on line 51. The call is accepted, the file no longer parses, and the model is not told. Had the call quoted its last line, `I='%I',`, the quote would not have matched line 51, and VERIFIED SELECTION would have refused the edit and reported that the quoted line is line 50.

Line numbers stay usable when the view is fresh. Line numbers go stale once the file changes, but the model keeps its view current. After GPT-5.6-luna's first successful refactor edit, it re-reads the file before almost every later LINE-RANGE edit, and none is rejected for a stale revision.

Result 2 (Figures 5 and 6). *Evidence makes errors visible:* quoting pays output for edits the harness can check, while pointing saves output but fails silently, mostly with a boundary one line off.

This trade-off is the same for every model, yet the form that wins is not.

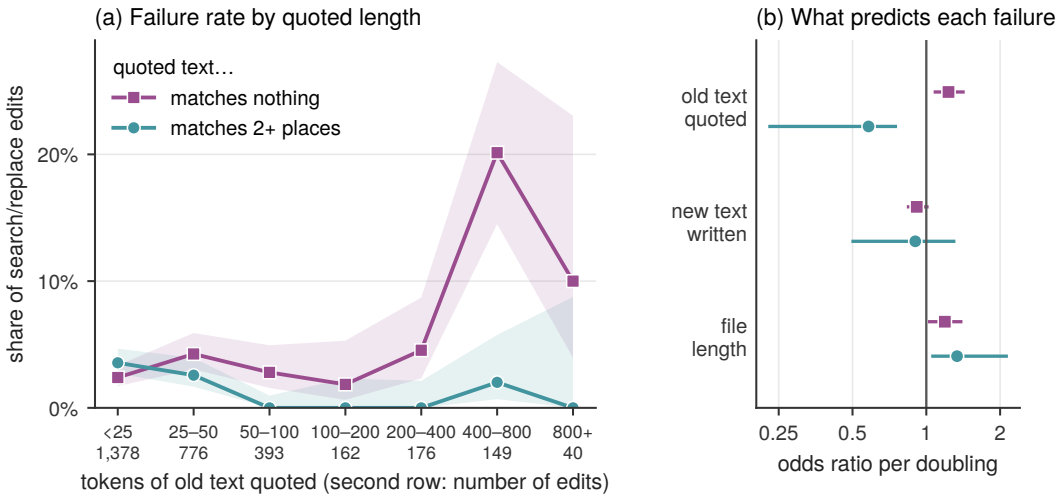


Figure 5: Why quoted targets fail. (a) Share of SEARCH/REPLACE edits whose old text matches nothing or matches more than once, by quoted length, with Wilson 95% bands. The second row of labels gives the number of edits. (b) Odds ratios per doubling from a logistic regression with model fixed effects, with intervals that resample whole tasks.

6 Result 3: Fit, Not Format

This is the model side of fit. We test whether acceptance depends on the pairing of model and interface, and whether the form a model reaches for is the form it writes best.

Acceptance depends on the pair. We regress first-edit acceptance on model, interface and task effects for the five models that ran all three interfaces on CanItEdit. Adding the model × interface interaction explains far more of the variation than the additive model ($p < 10^{-40}$; test statistics in Appendix B), so how reliably a model writes an interface's form depends on the pair. It holds for SEARCH/REPLACE and LINE-RANGE alone as well ($p = .003$), two interfaces that differ only in whether they point or quote. For task success the interaction is not significant ($p = .24$), since fit matters at the level of actions and the agent loop absorbs it before the outcome (Result 1).

No interface is best for every model. Acceptance rankings differ between models (Table 1). Line ranges have the highest first-edit acceptance for GPT-5.6-luna and DeepSeek-V4-Flash, SEARCH/REPLACE for GLM-5.3 and MiMo-v2.5-Pro, and patch for Qwen3.8-Flash. The gaps are large. On 62 CanItEdit tasks, GPT-5.6-luna's first line-range call is accepted and its first patch is not, and never the reverse. For Claude Sonnet 5, whose developer documents training it on string replacement, first edits are accepted at least as often with line ranges (1.00 against .92, p=.03). This is consistent with a training format being only one source of fluency, and with numbered reads making pointing easy to write.

Habit is not fluency. When one tool accepts all three forms, what a model reaches for first shows its habit, and that habit does not always match its fluency in this harness. GPT-5.6-luna, whose developer documents training on a patch format [28], picks PATCH more than twice as often as line ranges (35% against 16% of such runs; SEARCH/REPLACE takes the other 49%). Yet with PATCH alone its first patch is accepted in 23% of runs, and with LINE-RANGE alone its first line

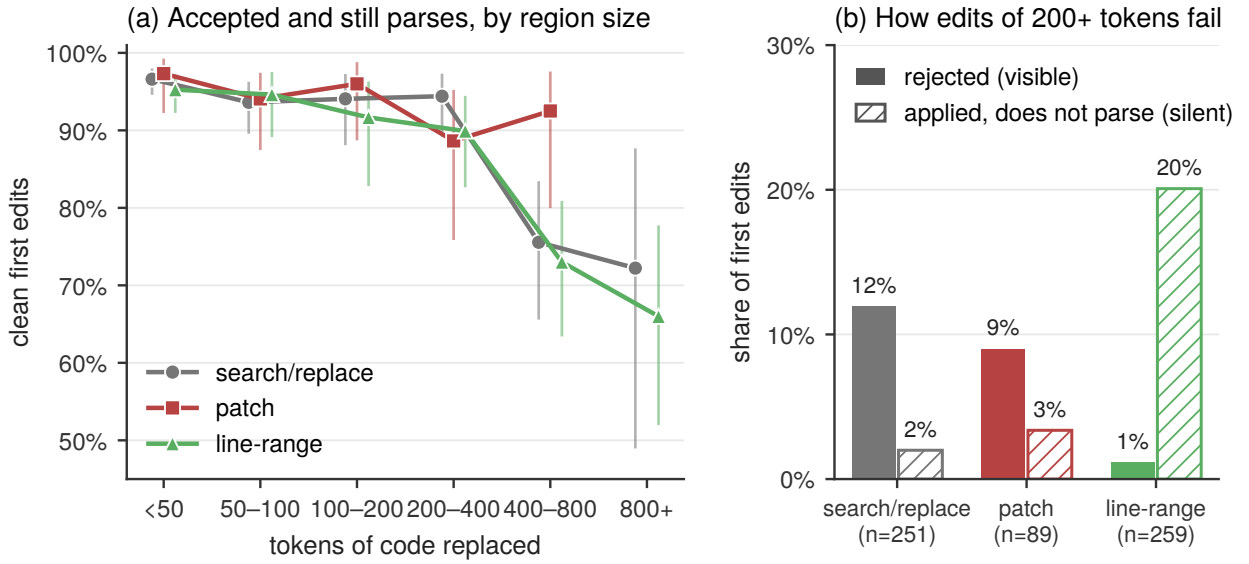


Figure 6: Outcomes of 2,339 replayed first targeted edits. (a) Share accepted and still parsing, by replaced-region size (Wilson 95% intervals, bins with fewer than 15 edits omitted). (b) For regions of 200 or more token-equivalents, edits rejected by the harness (solid, visible to the agent) and edits applied after which the file no longer parses (hatched, silent because no error reaches the agent, although a parser would catch them).

range in 99%. DeepSeek-V4-Flash goes further and reaches most often for the form it writes worst, while GLM-5.3 and MiMo-v2.5-Pro lean toward SEARCH/REPLACE, which is also their best-accepted form. A model's choice of interface thus reflects its habit, not a judgment of fit. This argues for one good default rather than a menu.

Packaging and position matter. When the three forms are offered together (Figure 7), no menu condition differs significantly in task success from the best single interface, but packaging changes first actions. Offering three named tools instead of one tool with three argument shapes raises GPT-5.6-luna's first-edit acceptance by 16 points (p=.004), because it then picks line ranges more often and patches less often. The tool listed first is more likely to be called first by GPT-5.6-luna and Qwen but not by MiMo-v2.5-Pro (Figure 7), a position bias also reported for tool selection in general [6]. Advice on when to use each form tends to raise first-edit acceptance and to lower output efficiency.

Result 3 (Table 1, Figure 7). *Fit, not format; habit is not fluency:* acceptance depends on the model and the interface together, and the form a model reaches for need not be the form it writes best.

Can one interface be cheap to write, checkable by the harness, and written well by every model? The next section builds one.

7 Designing for Shared Fluency: Verified Selection

VERIFIED SELECTION answers by asking only for the two forms every model is fluent in, the ones natural text shapes. Rather than betting on the fluency that one developer's post-training adds,

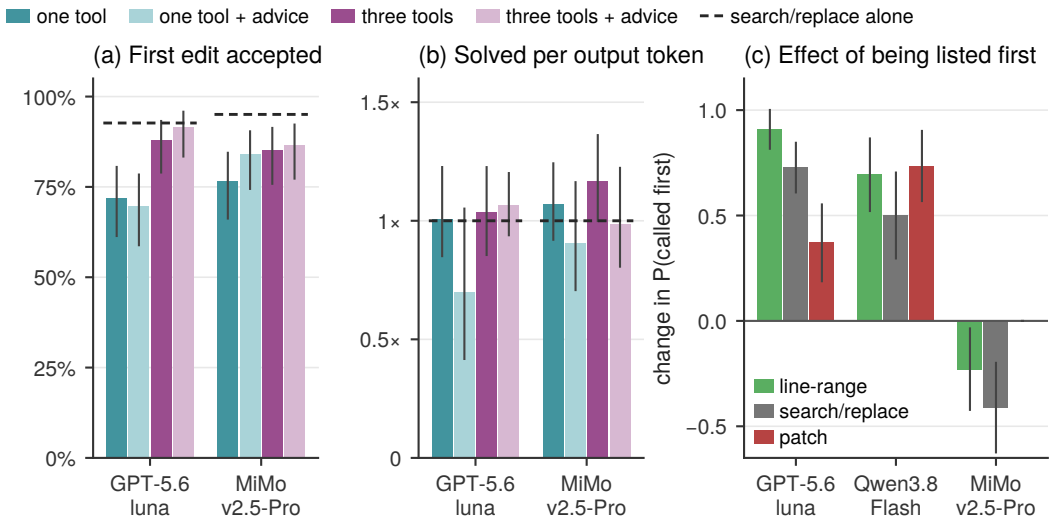


Figure 7: Packaging and position of the same three forms on CanItEdit. (a, b) One tool with three argument shapes or three named tools, with and without advice, against SEARCH/REPLACE alone (dashed), showing first-edit acceptance (Wilson 95% intervals) and tasks solved per output token (bootstrap 95% intervals). (c) Change in the probability that a tool is called first when it is listed first rather than later, in the three-tool menu (95% intervals).

it points with line numbers, as diagnostics, tracebacks and line links do, and quotes only the first and last line of the span, as diffs do. The model states its intent with an address and two lines of evidence, and the harness does the bookkeeping of checking, aligning and copying. Four design principles follow, one for each part of an edit's cost and risk.

P1: point with numbers, verify with boundaries. The model addresses the span by `start_line` and `end_line`, which draws on the fluency in pointing and keeps the restated evidence small. It also quotes the first and last line of the span as read (`first_line`, `last_line`), which draws on the fluency in quoting at the boundaries, where Result 2 locates the silent errors. The check accepts an edit only if both quotations match the file at the stated boundaries. This restores evidence at a cost of two lines per edit and turns a boundary error from silent into visible, lowering the silent-error rate.

P2: absorb small offsets when the evidence agrees. A strict check rejects many edits whose range is off by one line even though both quoted boundaries agree on where it should be, lowering acceptance. VERIFIED SELECTION therefore looks for the two quoted lines within a few lines of the stated boundaries and moves the range only when both boundaries agree on the move. A shift in which only one boundary would move across code is refused, and the refusal reports where the quoted text actually is. Every applied move is reported to the model as a note (exact rules in Appendix D).

P3: reference instead of retyping. A line of `new_text` consisting of `@@ref A-B@@` stands for the original lines A–B, in the numbering the model read, and `@@ref path:A-B@@` takes them from another file of the same call, so code can move between files without being retyped. The referenced lines must lie inside a range the call verifies, and the harness copies them before applying any edit

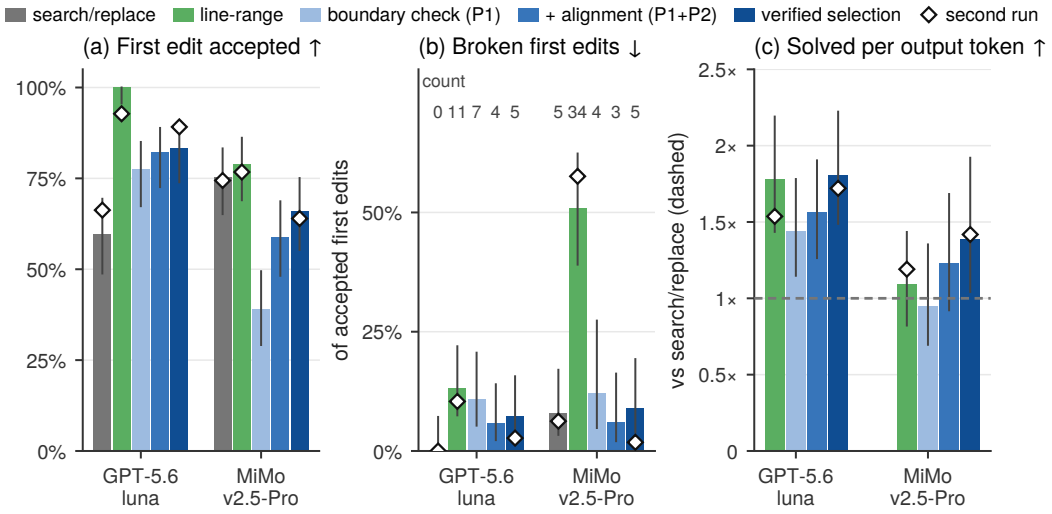


Figure 8: The ablation chain of Table 2, from SEARCH/REPLACE and bare LINE-RANGE through the boundary check (P1) and alignment (P1+P2) to VERIFIED SELECTION. (a) First edit accepted, (b) accepted first edits after which the file no longer parses (counts at the top), and (c) tasks solved per output token relative to SEARCH/REPLACE. Error bars are Wilson (a, b) or paired bootstrap (c) 95% intervals from the first run. White diamonds mark the second run of the SEARCH/REPLACE, LINE-RANGE and VERIFIED SELECTION arms.

(rules in Appendix D). For a move, this shrinks what the model writes from the whole block to one marker line (Figure 2e).

P4: one snapshot, no drift. All ranges in a call refer to the file as read and are placed before any edit is applied. Edits are then applied from the bottom up, so an insertion does not shift the line numbers of edits not yet applied [26].

The model-visible changes from LINE-RANGE are two string fields, a description of `@@ref` in the schema, and a paragraph of instructions with one move example.

8 Result 4: Evaluating Verified Selection

This section tests the three properties asked for above. We ask whether VERIFIED SELECTION stays cheap to write, whether it catches the errors that bare pointing lets through, and how models with opposite fits write it.

Table 2 and Figure 8 report VERIFIED SELECTION and its ablations on the refactors for GPT-5.6-luna and MiMo-v2.5-Pro, on the tasks where every arm has a valid run. Column P1 is the boundary check alone, P1+P2 adds alignment, and VERIFIED SELECTION adds `@@ref` (P3). Table 2 reports the first run of each arm. Table 3 repeats the SEARCH/REPLACE, LINE-RANGE and VERIFIED SELECTION arms with the same builds and settings, and the white diamonds in Figure 8 mark the second run.

Against search/replace: first edits go through. For GPT-5.6-luna, VERIFIED SELECTION gets the first edit through far more often than SEARCH/REPLACE (83% against 60% of runs, p=.001), and it is the most efficient interface for this model, at 1.77 times the tasks per output token over two runs (Table 3). MiMo-v2.5-Pro shows the other side of the trade-off. Its first edit is accepted

Table 2: VERIFIED SELECTION and its ablations on the large-file refactors, on tasks valid in every arm (GPT-5.6-luna 84, MiMo-v2.5-Pro 85). P1 is the boundary check, P1+P2 adds alignment, and VERIFIED SELECTION (P1–P3) adds `@@ref`. Efficiency is relative to SEARCH/REPLACE on the same tasks, and dollars assume perfect prefix caching (Equation (2)). *: paired 95% interval excludes 1×. A clean first edit is accepted and leaves a file that parses, although it may still change the wrong lines. First edit accepted is the share of runs whose first targeted edit is accepted, among runs that made one (all runs for GPT-5.6-luna, all but at most two per arm for MiMo-v2.5-Pro). Paired tests count a run without a targeted edit as not accepted.

	SEARCH/REPLACE	LINE-RANGE	P1	P1+P2	VERIFIED SELECTION
GPT-5.6-luna					
First edit accepted	.60	1.00	.77	.82	**.83**
Broken first edits (of applied)	0 of 50	11 of 84	7 of 65	4 of 69	**5 of 70**
Clean first edit	.60	.87	.69	.77	.77
Task success	.940	.952	.988	.964	.964
Output tokens / task (k)	4.4	2.5	3.2	2.9	2.5
Efficiency per output token	1	1.78*	1.44*	1.56*	**1.81***
Efficiency per dollar	1	1.34*	1.23*	1.30*	**1.41***
Runs using `@@ref`	–	–	–	–	56
MiMo-v2.5-Pro					
First edit accepted	.76	.81	.39	.60	.67
Broken first edits (of applied)	5 of 64	34 of 67	4 of 33	3 of 50	**5 of 56**
Clean first edit	.69	.39	.34	.55	**.60**
Task success	.929	.859	.894	.941	.929
Output tokens / task (k)	4.2	3.5	4.2	3.4	3.0
Efficiency per output token	1	1.09	0.94	1.23	**1.39***
Efficiency per dollar	1	0.96	0.96	1.12	1.20
Runs using `@@ref`	–	–	–	–	3

less often than with SEARCH/REPLACE in both runs, yet both runs are cheaper, at 1.43 times the tasks per output token (pooled) and equal or better success.

Against line ranges: silent failures become visible. Most first edits with bare line ranges are accepted, and many of them break the file. Over two runs, VERIFIED SELECTION cuts broken first edits from 19 to 7 for GPT-5.6-luna and from 71 to 6 for MiMo-v2.5-Pro (Table 3). For MiMo-v2.5-Pro, whose bare ranges break about half of the first edits they apply, this also raises task success (p=.002). The quoted boundaries also show how often a pointer misses. In the boundary-checked arms, 16% of first edits on the refactors and 8% on CanItEdit give line numbers that contradict their own quotes. A syntax check after each edit, as in SWE-agent's line-range command [35], would catch only the misses that break the file. That is about two thirds on the refactors and fewer than half on CanItEdit. The boundary check aligns or refuses all of them, needs no parser for the language, and tells the model where the quoted line actually is. For GPT-5.6-luna, whose bare line ranges were already mostly clean, VERIFIED SELECTION costs no efficiency.

Ablation. Read across the columns, Table 2 separates the two sources of gain. Pointing instead of quoting reduces what the model writes (LINE-RANGE against SEARCH/REPLACE), and VERIFIED SELECTION adds verification without adding output (VERIFIED SELECTION against LINE-RANGE). Each component addresses one part of an edit's cost or risk. The boundary check alone removes most broken edits but also refuses many recoverable ones, lowering first-edit acceptance and efficiency,

Table 3: Two runs of the headline arms on the refactors, on tasks valid in every arm of both runs (GPT-5.6-luna 82, MiMo-v2.5-Pro 85). Pooled combines both runs. "Per output token" is tasks solved per output token relative to SEARCH/REPLACE, with a bootstrap that resamples tasks together with both of their runs. Paired comparisons count, for each task, which arm did better over its two runs (tasks where the arms tie are left out) and use a sign test across tasks. A run without a targeted edit counts as not accepted. *: 95% interval excludes 1×.

	Run 1	Run 2	Pooled
GPT-5.6-luna (82 tasks)			
First edit accepted, SEARCH/REPLACE → VERIFIED SELECTION	.60 → .83	.67 → .90	33 vs 10 tasks, $p<.001$
Broken first edits, LINE-RANGE → VERIFIED SELECTION	11 → 5	8 → 2	15 vs 4 tasks, $p=.019$
Task success, LINE-RANGE → VERIFIED SELECTION	.939 → .963	.939 → .963	6 vs 2 tasks
Per output token, LINE-RANGE vs SEARCH/REPLACE	1.74	1.54	1.64 [1.39, 1.94]*
Per output token, VERIFIED SELECTION vs SEARCH/REPLACE	1.81	1.72	1.77 [1.48, 2.10]*
MiMo-v2.5-Pro (85 tasks)			
First edit accepted, SEARCH/REPLACE → VERIFIED SELECTION	.76 → .67	.79 → .66	20 vs 30 tasks
Broken first edits, LINE-RANGE → VERIFIED SELECTION	34 → 5	37 → 1	48 vs 2 tasks, $p<.001$
Task success, LINE-RANGE → VERIFIED SELECTION	.859 → .929	.812 → .941	20 vs 4 tasks, $p=.002$
Per output token, LINE-RANGE vs SEARCH/REPLACE	1.09	1.19	1.14 [0.90, 1.46]
Per output token, VERIFIED SELECTION vs SEARCH/REPLACE	1.39	1.47	1.43 [1.10, 1.93]*

most sharply for MiMo-v2.5-Pro. Alignment recovers both without readmitting broken edits. `@@ref` then shortens the output where the model uses it. GPT-5.6-luna references lines in two thirds of its runs and becomes 1.16 times as efficient per output token as with alignment alone, while MiMo-v2.5-Pro rarely uses it.

Small edits. On CanItEdit, where most edits touch a few short lines, VERIFIED SELECTION changes efficiency little, and `@@ref` is not used. Little is restated in small edits, so there is little to save. For GPT-5.6-luna, task success with VERIFIED SELECTION is higher than with SEARCH/REPLACE in both of two runs ($p=.02$) at similar efficiency.

Claude Sonnet 5. Claude's developer documents training it on exact string replacement [3, 4], while VERIFIED SELECTION did not exist before this study. On CanItEdit, Claude edits in place on its first try in 78 of 80 runs with VERIFIED SELECTION against 70 of 80 with SEARCH/REPLACE ($p=.02$). With SEARCH/REPLACE, the other ten runs either have their edit rejected (6) or rewrite the whole file instead (4), so its first targeted edit is accepted in 70 of 76 runs, the .92 of Table 1. With VERIFIED SELECTION, every run edits in place, and no applied first edit breaks the file in either arm. VERIFIED SELECTION solves 1.25 times as many tasks per output token as SEARCH/REPLACE, at similar task success. An interface no model was trained on is thus accepted at least as often as the one this model was trained on.

Edits across files. RefactorBench refactors span files, and nearly every accepted first line-range edit changes more than one file in a single call. The pattern of the large-file refactors holds (Table 4). Task success is again flat while first-edit acceptance varies widely. Bare line ranges break 11 accepted first edits and VERIFIED SELECTION 2 ($p=.012$), and the quoting interfaces break none. VERIFIED SELECTION sometimes moves code between files with `@@ref`, and its efficiency gain over SEARCH/REPLACE concentrates on the 34 tasks that move code (1.25 times per output token), while on renames and signature changes bare line ranges are the most efficient interface.

Table 4: RefactorBench, Microsoft's multi-file refactorings, with one run of each arm by GPT-5.6-luna on 84 tasks. Efficiency is relative to SEARCH/REPLACE on the same tasks, per output token and per billed dollar; *: paired 95% interval excludes 1×. Broken first edits are replayed on the repository (Appendix E).

	SEARCH/REPLACE	PATCH	LINE-RANGE	VERIFIED SELECTION
Task success	.750	.774	.750	.786
First edit accepted	.61	.39	.93	.80
Broken first edits (of applied)	0 of 51	0 of 33	11 of 78	**2 of 67**
Rejected edit calls	39%	57%	7%	21%
Efficiency per output token	1	1.02	1.26*	1.18
Efficiency per dollar	1	1.00	1.09	1.19

Result 4 (Tables 2, 3 and 4). *Pointing checked at its boundaries keeps most of the efficiency of pointing and removes most of its silent errors,* for three models and for edits that span files.

9 Discussion: Designing Edit Interfaces

Our findings suggest two practices for edit interfaces.

Measure the loop, not only the outcome. An agent stops when the model judges its work done, so an interface defect rarely shows as a failed task. It shows as the tokens spent getting there. Reporting first-action acceptance and tasks solved per output token alongside success, within each model, is what exposes it, in line with calls for cost-controlled agent evaluation [20, 21]. Dollar comparisons should state their caching assumption, because providers billed as cached a much smaller share of input than an append-only context allows.

Divide the edit call. Which form a model writes reliably differs between models (Section 6), and left to choose, models follow their habit even where it serves them poorly, so a menu of formats is no substitute for a good default. A harness that serves many models can provision each with its own format [16, 19], or offer one interface in the forms they all write fluently. The first route chases each model's best acceptance. The second draws on the fluency that natural data shapes and every model shares, rather than on what one developer's post-training adds. It gives up some acceptance for some models (MiMo-v2.5-Pro's first edit goes through less often with VERIFIED SELECTION than with SEARCH/REPLACE) in exchange for errors that are visible rather than silent, trading part of the model side of fit for the system side. Either way, a model's output is the most expensive and least exact part of an edit, so the interface should ask the model only for its intent (a pointer and the least evidence that makes the pointer checkable) and leave the rest to the harness. Numbering the file, checking boundaries, shifting a range and copying code are exact and almost free for a harness. Doing them there moves work out of output tokens, which are decoded one at a time, into steps that read what is already in the context, which the KV cache makes cheap [39]. Revision hashes already follow this pattern for stale views, and boundary checks extend it to miscounted positions.

10 Related Work

Edit representations. Aider compared search/replace blocks and unified diffs and introduced the refactoring benchmark used here [1, 15]. CanItEdit and EDIT-Bench evaluate instructed edits [9, 11], and RefactorBench multi-file refactoring [14]. Diff-XYZ evaluates generating and applying diffs outside an agent loop [18]. Cheng et al. replace offset-based diffs with block-level rewrites of syntactic units (BlockDiff, FuncDiff) and train models to choose between a diff and the full file (AdaEdit) [10]. SWE-Edit moves editing into a separate editor subagent trained to choose between find-and-replace and whole-file rewriting [38]. A comparison of five edit tools in one agent covers 23 SWE-bench Pro tasks [36], and practitioner harnesses keep adding variants, such as search blocks with line-number hints [31] and line references tagged with content hashes [7]. Lee et al. measure the cost of structured output, the "format tax" [23]. OctoPack studied line-number diffs as a training target [26]. These works ask which representation of a change, or which trained choice among them, gives the best final result. We vary only how an edit call names its target, and measure what final results hide, namely first-action acceptance, the cost of the loop's recovery, and whether an error is seen at all. VERIFIED SELECTION needs no training. It changes what the harness checks.

Agent–computer interfaces and harnesses. SWE-agent showed that interface design changes agent performance and used a line-range edit command that rejects edits introducing syntax errors [35]. Surveys treat the harness as part of the agent [12, 37], and DCAS narrows the gap between training and deployment scaffolds by fine-tuning on planning-aware trajectories [32]. We measure fit at one component, the edit interface, and improve it in the harness rather than in training.

Cost and serving of agents. Cost-controlled evaluation [20, 21] motivates our efficiency measures. Serving research exploits the append-only structure of agent contexts through prefix caching and KV reuse [22, 25, 39]. Recent work characterizes coding-agent traces and the effect of caching on agent cost [24, 41]. We tie these costs to an interface decision in the harness.

11 Conclusion

An edit interface decides how much of the target a model must restate, and therefore what the agent loop pays and what the harness can verify. Task success hides misfit, and the loop pays for it in retries. Quoting buys checkable edits with output, while pointing saves output and fails silently. How reliably a model writes each form depends on the pair, and a model's habit is not its fluency. VERIFIED SELECTION asks only for the two forms every model is fluent in and keeps most of both benefits on large edits and across files. The lesson for edit interfaces is fit, not format.

Limitations. We count a silent error only when it leaves a trace, that is, when the file no longer parses or when line numbers contradict the quotes of a call that quotes its boundaries. Misplaced edits that still parse are therefore undercounted for bare line ranges. We measure fluency and habit, not their origin; their link to pretraining text and to post-training harness trajectories rests on documentation. The large-file results come from two models, with the three headline interfaces run twice, and the multi-file results from one model in one run. The properties of the interfaces carry over to new models. A line range carries no evidence a harness could check, and a quotation costs more output and fails more often as it grows. How often each form is written correctly depends on the model and has to be measured for each.

Appendix

A Setup and Replay Details

Run limits and file views. A run ends when the model stops or reaches 48 model calls, 64 tool calls, or one million tokens. Each `read_file` result carries a 16-character file revision. After GPT-5.6-luna's first successful refactor edit, it re-reads the file before 50 of its 53 later LINE-RANGE edits.

Table 5: Model settings. Reasoning is the provider default unless noted.

Model	Sampling	Reasoning
GPT-5.6-luna	provider defaults[‡]	on
GLM-5.3	temperature 1.0, top-p 0.95	on
DeepSeek-V4-Flash	temperature 1.0, top-p 1.0	off (provider default)
MiMo-v2.5-Pro	harness default temperature, top-p 0.95	on
Qwen3.8-Flash	temperature 1.0, top-p 0.95	on
Claude Sonnet 5	provider defaults[‡]	adaptive (provider default)

[‡]The endpoint does not accept a temperature parameter.

Invalid runs. A run is invalid when the provider or the harness fails rather than the model (upstream HTTP errors, a harness process that exits without finishing the turn). Runs that failed before any model response (e.g., on rate limits) were retried with the same build. Seven of the 1,928 runs behind Table 1 remained invalid. Claude Sonnet 5 has no PATCH arm because Anthropic's API rejects the tool schema of the PATCH interface (a top-level `oneOf`).

Replay of first edits. We replay 2,339 first targeted calls with interpretable addresses in the main study, leaving out calls rejected for their argument shape, patch grammar or a stale revision. Each action is applied to the original file, the replaced region is estimated in token-equivalents (characters divided by four), and the result is parsed. Replay agrees with the harness on all but 11 calls.

Dollar regimes. Because the method arms ran after the baselines, on dates with different cache hit rates, we compute dollars as billed, with no caching, and with perfect prefix caching, using per-model token prices recovered by least squares from per-call billing. For a run τ whose t-th model call has prompt length P_t and output O_t, perfect prefix caching gives

$$C(\tau) = \sum_t \Big[p_{\text{cache}}\, P_{t-1} + p_{\text{in}}\, (P_t - P_{t-1}) + p_{\text{out}}\, O_t \Big]. \tag{2}$$

B Test Statistics and Exact Counts

We write $+a/-b$ for the discordant tasks of a paired comparison. Bootstrap intervals use at least 2,000 resamples.

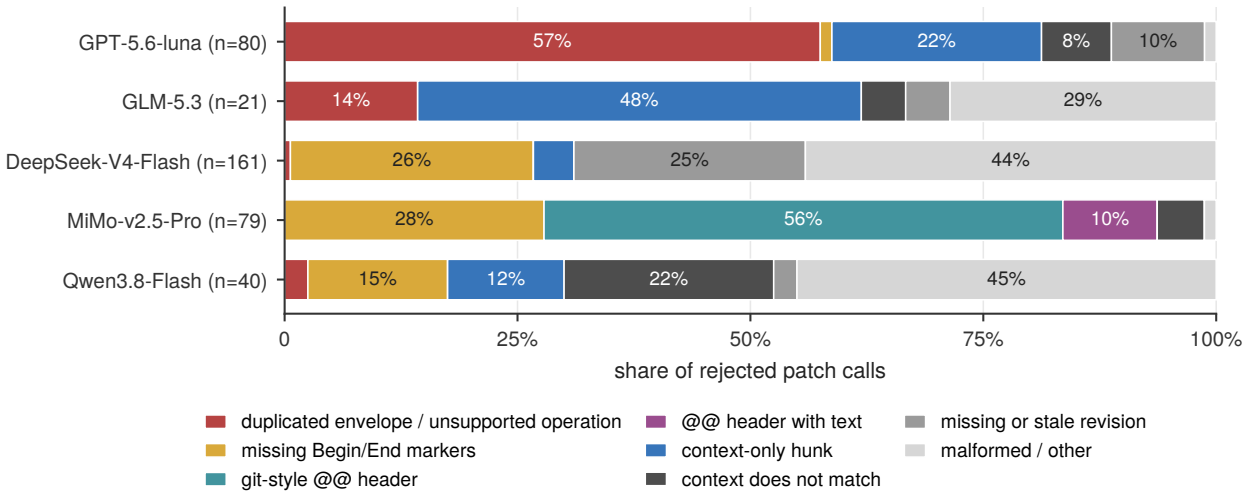


Figure 9: Rejected patch calls on CanItEdit, by type.

Quotation length (Result 2). Across 3,074 SEARCH/REPLACE edits, doubling the quoted old text has an odds ratio of 1.23 for a no-match rejection (95% interval [1.07, 1.43]), doubling the new text 0.91 [0.83, 1.02], and longer quotations reduce ambiguous matches (0.58 [0.23, 0.76]).

Region size (Result 2). For edits replacing fewer than 200 token-equivalents, 510 of 539 LINE-RANGE calls (.946) and 872 of 913 SEARCH/REPLACE calls (.955) apply and leave a file that parses. For larger regions, SEARCH/REPLACE rejects 12.0% and applies 2.0% that break the file, LINE-RANGE rejects 1.2% and applies 20.1% that break it, and PATCH rejects 9.0% and applies 3.4% that break it. Of the 74 broken first LINE-RANGE edits, 46 (62%) parse after moving one boundary by a single line, 21 (28%) have the parser error within model-written text, and 7 show neither pattern.

Pairing (Result 3). Adding the model × interface interaction to the logistic regression of first-edit acceptance gives a likelihood ratio of 239.7 (8 df, $p < 10^{-40}$; deviance 909.6 → 669.9 over 1,090 runs). Restricted to SEARCH/REPLACE and LINE-RANGE it is 16.2 (4 df, $p = .003$), and for task success 10.4 (8 df, $p = .24$). For Qwen3.8-Flash, patch beats line range on 37 tasks and loses on seven. Claude Sonnet 5's line-range advantage over SEARCH/REPLACE is +6/−0.

Rejected patches (Result 3). Figure 9 breaks down the rejected patch calls on CanItEdit by type. Rejected patches differ by model in kind, not only in rate. MiMo-v2.5-Pro mixes git-diff and Codex formats in one patch, and GPT-5.6-luna often duplicates the patch envelope, errors that carry one harness's format into another and that the reference `apply_patch` rejects too.

Menus (Result 3). Three named tools instead of one tool with three argument shapes raise GPT-5.6-luna's first-edit acceptance by 15.9 points (+16/−3, p=.004); it then picks PATCH in 12% of runs instead of 35% and line ranges in 37% instead of 16%. Advice on when to use each form raises first-edit acceptance in six of seven comparisons and lowers output efficiency in five.

Verified selection (Result 4). For GPT-5.6-luna, VERIFIED SELECTION leaves the first edit clean more often than SEARCH/REPLACE (77% against 60%, p=.014). The boundary check alone gives first-edit acceptance of .77 (GPT-5.6-luna) and .39 (MiMo-v2.5-Pro) and efficiency of 1.44 and

0.94; alignment raises them to .82 and .60, and 1.56 and 1.23. GPT-5.6-luna references lines in 56 of 84 runs and is 1.16 times as efficient per output token as with alignment alone ([1.03, 1.29]); MiMo-v2.5-Pro uses `@@ref` in 3 of 85 runs and is 1.13 times as efficient ([0.96, 1.32]). Of 667 first edits in the boundary-checked arms, 107 give line numbers that contradict their quotes by one to three lines; as bare ranges, 70 would have broken the file and 37 would have parsed. On CanItEdit, 24 of 313 first edits contradict their quotes, of which 10 would have broken the file and 14 would have parsed. On CanItEdit, VERIFIED SELECTION is 1.02 (GPT-5.6-luna) and 0.98 (MiMo-v2.5-Pro) times as efficient as SEARCH/REPLACE per output token; for GPT-5.6-luna, pooled task success is .810 against .747, and 11 of the 13 tasks whose outcome differs favor VERIFIED SELECTION (p=.02), at 1.13 times the efficiency pooled ([0.88, 1.48]). For Claude Sonnet 5, SEARCH/REPLACE has four whole-file rewrites and six rejected targeted edits among its first edits (+9/−1 for VERIFIED SELECTION); VERIFIED SELECTION solves 1.25 times as many tasks per output token ([1.00, 1.61]) and 1.11 times as many per dollar under perfect caching ([0.93, 1.34]), with task success .775 against .787 (+2/−3).

C Cost Breakdown

Our benchmarks' contexts are about 2 to 7 thousand tokens per model call on average. Under perfect prefix caching, re-reading accounts for 1–17% of a run's cost, newly read input for 25–64%, and output for 33–69%. For GPT-5.6-luna on the refactors, output is 51% of the SEARCH/REPLACE arm's cost and 38% of the LINE-RANGE arm's, and the line-range arm is cheaper overall without reading more. In every arm of GPT-5.6-luna and MiMo-v2.5-Pro, 68–86% of input tokens were a reusable prefix, yet across providers and dates, providers billed 18–82% of input tokens as cached, so billed caching fell short of what an append-only context permits. In about 4,300 Claude Code and Codex sessions, the median model call re-read 119K cached tokens, appended 875 new ones and generated 214 [41].

D Alignment and Reference Rules of Verified Selection

VERIFIED SELECTION searches for the two quoted lines within ±3 lines of the stated boundaries and moves the range when both boundaries move consistently, that is, when both quoted lines are found the same number of lines from their stated boundaries, or at distances that differ only by intervening blank lines. A shift in which only one boundary would move across code is refused, and the refusal reports where the quoted text actually is. If the new text's first or last line repeats the non-blank line just outside the range, indentation included, the range is widened to replace that line instead of duplicating it.

A `@@ref A-B@@` marker stands for lines A–B in the numbering the model read, re-indented so that their shallowest line sits at the marker's indentation. The lines must lie inside a verified range of the same call (for `@@ref path:A-B@@`, a verified range in another file of the call), and that range's alignment shift is applied to them. Markers are expanded once, by the harness, before any edit is applied. The expansion is a copy, not a live reference, so later edits cannot change its meaning. The `path` form was added for the multi-file runs (Appendix E).

E RefactorBench Setup

RefactorBench [14] has 100 tasks in nine Python repositories, each graded by AST tests on the final repository. We use each task's descriptive instruction as the prompt and its released tests

unchanged. The 16 tasks whose tests require a file to be deleted are left out, because the edit tools can create and rewrite files but not delete them. On the untouched repositories, none of the 100 tasks passes its tests, and a hand-made solution to one rename task passes.

Multi-file tasks need more reading than the harness's defaults allow. In a pilot, runs in every arm ended before their first edit. For all arms alike we therefore raise the limits to 256 tool calls, 96 model calls and 20 million tokens per run, and keep every tool result in the context (the harness otherwise keeps only the six most recent in full). The `@@ref path:A-B@@` form of P3 was added for these runs.

The replay applies every accepted call of a run, in order, to a virtual copy of the repository with the harness's rules for each form, and reproduces the final repository of all 336 runs. A task counts as moving code when its instruction asks to move, transfer, relocate, combine, merge, consolidate, extract, encapsulate, take out or shift code (34 tasks). Of the 78 accepted first LINE-RANGE edits, 73 change more than one file in a single call. Broken first edits differ between LINE-RANGE and VERIFIED SELECTION on 10 tasks against 1. VERIFIED SELECTION uses `@@ref` across files in 7 runs, and on the 34 tasks that move code it is 1.25 times as efficient as SEARCH/REPLACE per output token ([1.03, 1.53]).

Acknowledgments

We thank Wei Deng and Yinling Zhang for comments on drafts of this paper, and the contributors to S-Code, the open-source harness on which all experiments run.

Use of AI Tools

We used an AI coding assistant (Claude Code) to implement the harness modifications and verified selection, to run and log the experiments, to write the analysis and replay scripts, and to help with the literature search and the writing. The authors reviewed all of it; every number is computed by the scripts from the logged runs, and every cited source was checked against its primary text.

References

[1] Aider-AI. Aider refactoring benchmark. https://github.com/Aider-AI/refactor-benchmark, 2023.

[2] Anthropic. Prompt caching. Claude documentation, https://platform.claude.com/docs/en/build-with-claude/prompt-caching, 2026. Accessed 2026-09-25.

[3] Anthropic. Text editor tool. Claude documentation, https://platform.claude.com/docs/en/agents-and-tools/tool-use/text-editor-tool, 2026. Accessed 2026-10-02.

[4] Anthropic. Tool use with Claude. Claude documentation, https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview, 2026. Accessed 2026-10-02.

[5] Anthropic. Claude Code tools reference. https://code.claude.com/docs/en/tools-reference, 2026. Accessed 2026-09-25.

[6] Thierry Blankenstein, Jialin Yu, Zixuan Li, Vassilis Plachouras, Sunando Sengupta, Philip Torr, Yarin Gal, Alasdair Paren, and Adel Bibi. Biasbusters: Uncovering and mitigating tool selection bias in large language models. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.00307.

[7] Can Bölük. We improved 15 LLMs at coding in one afternoon. Only the harness changed. Blog post, https://blog.can.ac/2026/02/12/the-harness-problem/, February 2026. Accessed 2026-10-01.

[8] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. Qwen3-coder-next technical report, 2026. URL https://arxiv.org/abs/2603.00729.

[9] Federico Cassano, Luisa Li, Akul Sethi, Noah Shinn, Abby Brennan-Jones, Jacob Ginesin, Edward Berman, George Chakhnashvili, Anton Lozhkov, Carolyn Jane Anderson, and Arjun Guha. Can it edit? evaluating the ability of large language models to follow code editing instructions, 2024. URL https://arxiv.org/abs/2312.12450.

[10] Wei Cheng, Yongchang Cao, Chen Shen, Binhua Li, Jue Chen, Yongbin Li, and Wei Hu. To diff or not to diff? structure-aware and adaptive output formats for efficient llm-based code editing. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. URL https://arxiv.org/abs/2604.27296.

[11] Wayne Chi, Valerie Chen, Ryan Shar, Aditya Mittal, Jenny Liang, Wei-Lin Chiang, Anastasios Nikolas Angelopoulos, Ion Stoica, Graham Neubig, Ameet Talwalkar, and Chris Donahue. Edit-bench: Evaluating llm abilities to perform real-world instructed code edits, 2025. URL https://arxiv.org/abs/2511.04486.

[12] Guanting Dong, Xiaoshuai Song, Yuyang Hu, Jiajie Jin, Chenghao Zhang, Yifei Chen, Xiaoxi Li, Huaying Yuan, Xinyu Yang, Tongyu Wen, Jiejun Tan, Hongjin Qian, Shijue Huang, Junting Lu, Zhenyu Li, Wanjun Zhong, Yutao Zhu, Tat-Seng Chua, Zhicheng Dou, and Ji-Rong Wen. Towards long-horizon agents: A survey. Preprints.org preprint, July 2026. URL http://dx.doi.org/10.20944/preprints202607.1328.v1.

[13] Free Software Foundation. GNU coding standards: Formatting error messages. https://www.gnu.org/prep/standards/html_node/Errors.html. Accessed 2026-09-25.

[14] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. Refactorbench: Evaluating stateful reasoning in language agents through code, 2025. URL https://arxiv.org/abs/2503.07832.

[15] Paul Gauthier. Unified diffs make GPT-4 Turbo 3x less lazy. https://aider.chat/2023/12/21/unified-diffs.html, 2023.

[16] Paul Gauthier. Edit formats. Aider documentation, https://aider.chat/docs/more/edit-formats.html, 2026. Accessed 2026-09-22.

[17] GitHub. Creating a permanent link to a code snippet. GitHub Docs, https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-a-permanent-link-to-a-code-snippet, 2026. Accessed 2026-09-25.

[18] Evgeniy Glukhov, Michele Conti, Egor Bogomolov, Yaroslav Golubev, and Alexander Bezzubov. Diff-xyz: A benchmark for evaluating diff understanding, 2025. URL https://arxiv.org/abs/2510.12487.

[19] Stefan Heule and Jediah Katz. Continually improving our agent harness. Cursor engineering blog, https://cursor.com/blog/continually-improving-agent-harness, April 2026. Accessed 2026-09-22.

[20] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for ai agent evaluation, 2025. URL https://arxiv.org/abs/2510.11977.

[21] Sayash Kapoor, Benedikt Stroebl, Zachary S Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=Zy4uFzMviZ.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626. ACM, October 2023. doi: 10.1145/3600006.3613165. URL http://dx.doi.org/10.1145/3600006.3613165.

[23] Ivan Yee Lee, Loris D'Antoni, and Taylor Berg-Kirkpatrick. The format tax, 2026. URL https://arxiv.org/abs/2604.03616.

[24] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks, 2026. URL https://arxiv.org/abs/2601.06007.

[25] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An

efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2443–2459, Renton, WA, May 2026. USENIX Association. ISBN 978-1-939133-54-0. URL https://www.usenix.org/conference/nsdi26/presentation/luo.

[26] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.07124.

[27] OpenAI. Codex CLI. https://github.com/openai/codex, 2025.

[28] OpenAI. GPT-4.1 prompting guide. OpenAI Cookbook, https://cookbook.openai.com/examples/gpt4-1_prompting_guide, 2025. Accessed 2026-09-22.

[29] OpenAI. Prompt caching. OpenAI API documentation, https://developers.openai.com/api/docs/guides/prompt-caching, 2026. Accessed 2026-09-25.

[30] Python Software Foundation. The Python tutorial: Errors and exceptions. https://docs.python.org/3/tutorial/errors.html. Accessed 2026-09-25.

[31] Roo Code. apply_diff. Roo Code documentation, https://docs.roocode.com/advanced-usage/available-tools/apply-diff, 2026. Accessed 2026-10-01.

[32] Kishanthan Thangarajah, Boyuan Chen, and Ahmed E. Hassan. DCAS: Decoupling CLI agent scaffolding to internalize planning across scaffolds, 2026. URL https://arxiv.org/abs/2608.06113.

[33] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.18449.

[34] Xiaomi LLM-Core Team. MiMo-V2-Flash technical report, 2026. URL https://arxiv.org/abs/2601.02780.

[35] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

[36] yaoyi1222. edit-tool-bench. https://github.com/yaoyi1222/edit-tool-bench, 2026.

[37] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhong-Zhi Li, Xiangyuan Xue, Yijiang Li, Yifan Zhou, Yang Chen, Chen Zhang, Yutao Fan, Zihu Wang, Songtao Huang, Francisco Piedrahita Velez, Yue Liao, Hongru WANG, Mengyue Yang, Heng Ji, Jun Wang, Shuicheng YAN, Philip Torr, and LEI BAI. The landscape of agentic reinforcement learning for LLMs: A survey. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL https://openreview.net/forum?id=RY19y2RI1O. Survey Certification.

[38] Yikai Zhang, Jiaxin Pei, Kenan Li, Qirui Jin, Maoquan Wang, Jin Pan, Yu Kang, Shengyu Fu, Elsie Nallipogu, Junjie Hu, Yufan Huang, and Zijian Jin. Swe-edit: Rethinking code editing for efficient swe-agent, 2026. URL https://arxiv.org/abs/2604.26102.

[39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 62557–62583. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-2000. URL http://dx.doi.org/10.52202/079017-2000.

[40] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[41] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Tracelab: Characterizing coding agent workloads for llm serving, 2026. URL https://arxiv.org/abs/2606.30560.